



UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
"Conoceréis la verdad y la verdad os hará libres"

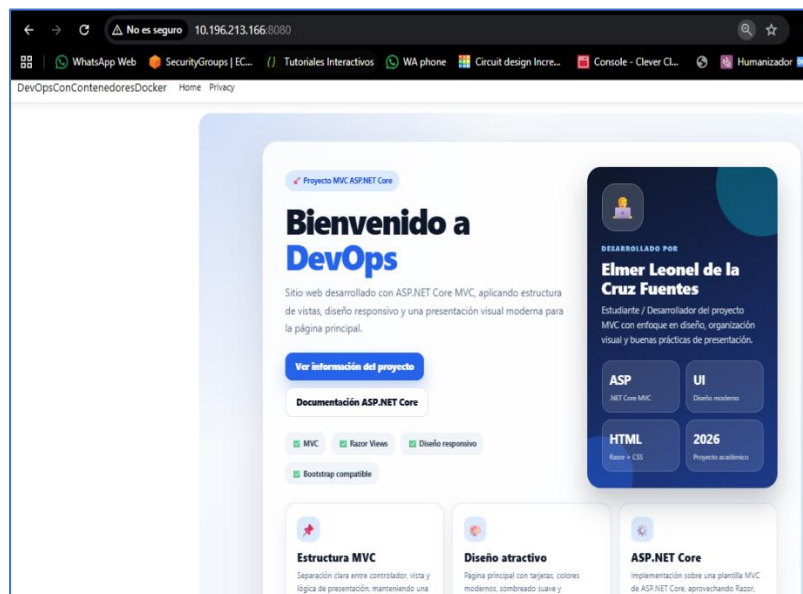
**INGENIERIA EN SISTEMAS DE INFORMACION Y CIENCIAS DE
LA COMPUTACION**

Nombre: Elmer Leonel De La Cruz Fuentes

Carne: 7490-23-23907

Actividad

Contenedores Docker



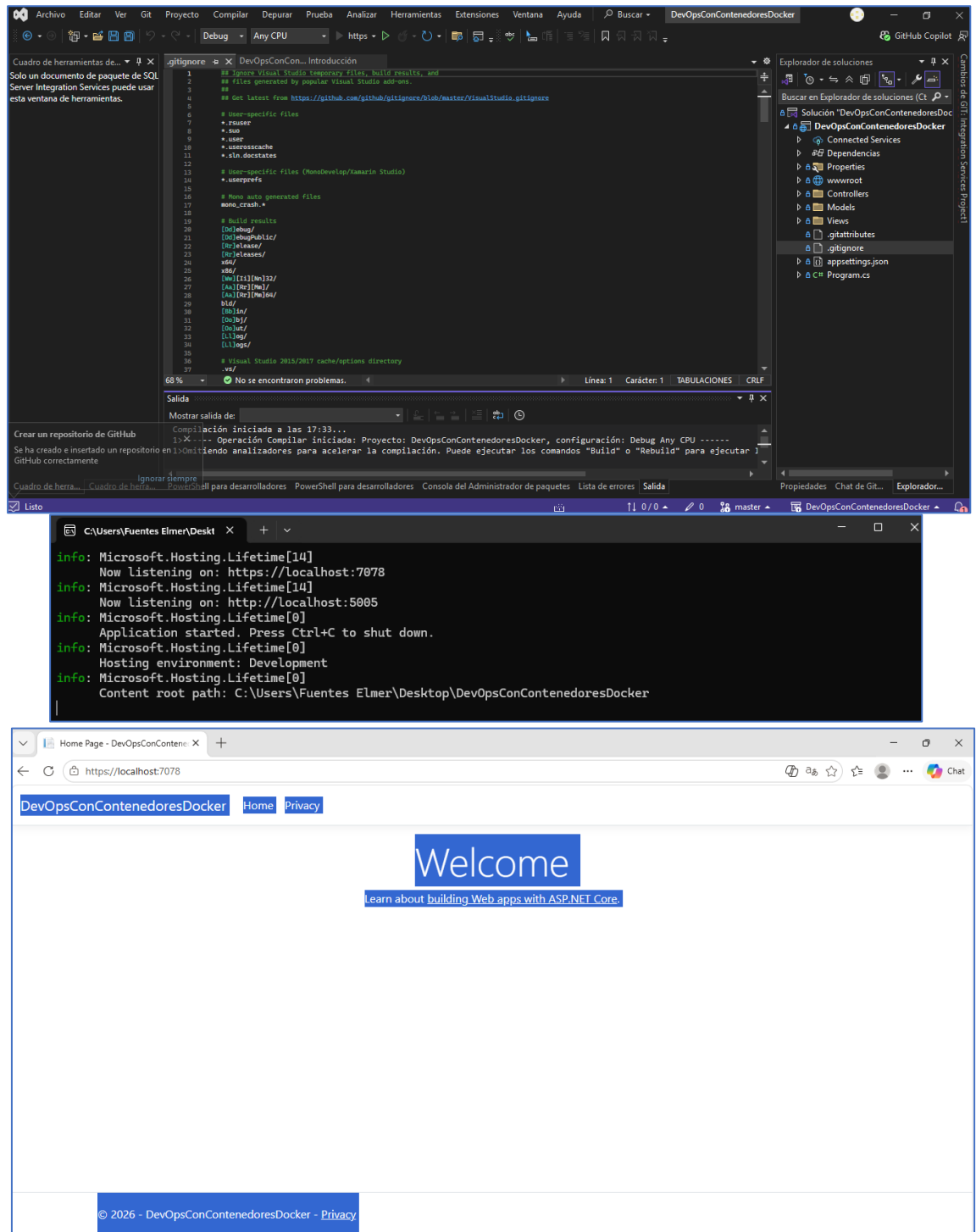
Asignatura: SISTEMAS OPERATIVOS II

Ing. Luis Gutiérrez

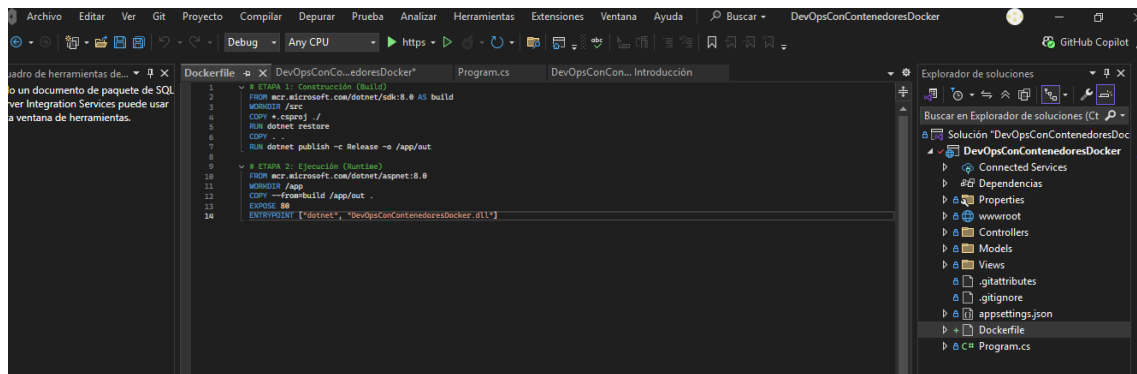
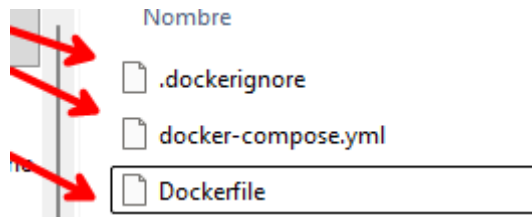
Fase 1: Creación y Ejecución Local

Para garantizar el punto aquí, asegúrate de documentar la estructura de archivos que genera dotnet.

- **Comando:** dotnet new mvc -n ProyectoDockerLab
- **Evidencia:** Captura de pantalla de la ejecución local (dotnet run) funcionando en <http://localhost:5000> o 5001.



Fase 2: Dockerfile



Fase 3: Gestión Con Git

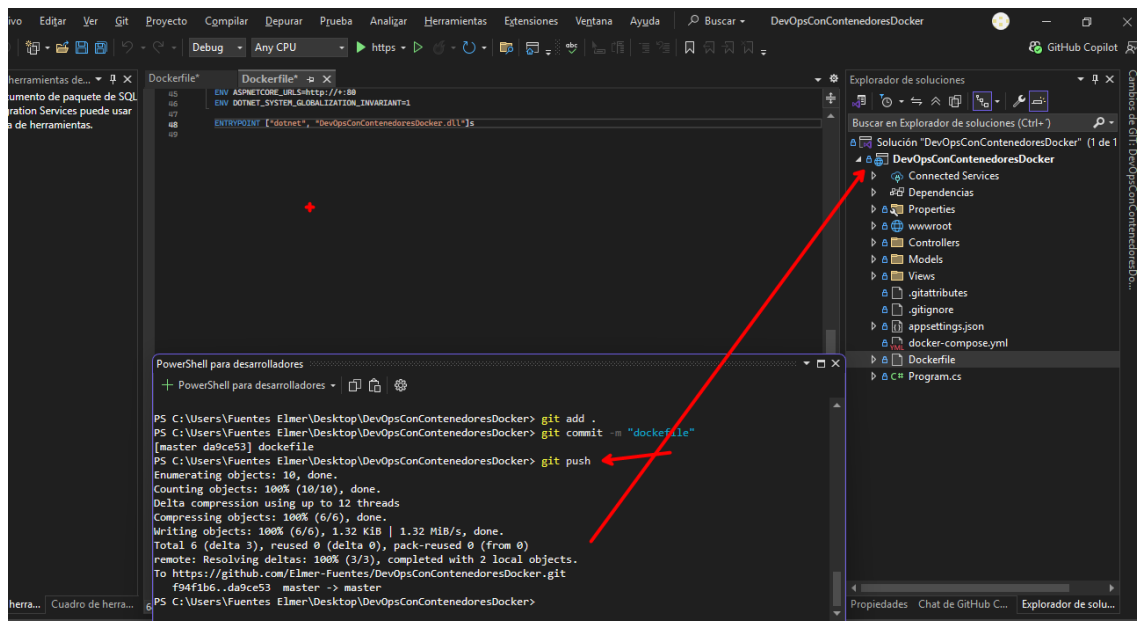
git init

git add .

git commit -m "Laboratorio Docker Tarea 6"

git remote add origin

git push -u origin main



Fase 4. Clonar repositorio en la VM (Centos)

```
CentOS [Corriendo] - Oracle VirtualBox : 1
31 de may 20:23
root@localhost:/home/elmerfuentes -- bash
~/DevOpsConContenedoresDocker
elmerfuentes@localhost:~/DevOpsConContenedoresDocker root@localhost:/home

root@localhost:/home/elmerfuentes# git clone https://github.com/Elmer-Fuentes/DevOpsConContenedoresDocker.git
Clonando en 'DevOpsConContenedoresDocker'...
remote: Enumerating objects: 121, done.
remote: Counting objects: 100% (121/121), done.
remote: Compressing objects: 100% (74/74), done.
remote: Total 121 (delta 43), reused 118 (delta 40), pack-reused 0 (from 0)
Recibiendo objetos: 100% (121/121), 867.54 KiB | 1.66 MiB/s, listo.
Resolviendo deltas: 100% (43/43), listo.
root@localhost:/home/elmerfuentes#
```

```
CentOS [Corriendo] - Oracle VirtualBox : 1
4 de jun 06:26
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker -- sudo su -

root@localhost:/home/elmerfuentes# # Clonar el repositorio nuevamente
git clone https://github.com/Elmer-Fuentes/DevOpsConContenedoresDocker.git

# Entrar a la nueva carpeta recién clonada
cd DevOpsConContenedoresDocker
Clonando en 'DevOpsConContenedoresDocker'...
remote: Enumerating objects: 154, done.
remote: Counting objects: 100% (154/154), done.
remote: Compressing objects: 100% (96/96), done.
Recibiendo objetos: 100% (154/154), 871.73 KiB | 538.00 KiB/s, listo.
remote: Total 154 (delta 62), reused 142 (delta 50), pack-reused 0 (from 0)
Resolviendo deltas: 100% (62/62), listo.
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

Entra a la carpeta del proyecto (donde se descargó el código):

`cd ~/DevOpsConContenedoresDocker`

```
root@localhost:/home/elmerfuentes# cd DevOpsConContenedoresDocker
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# SS
```

1. Construcción de la imagen Docker

Desde la carpeta raíz del proyecto, donde se encuentra el archivo Dockerfile, se ejecutó el siguiente comando:

docker build -t devopscontenedoresdocker:v1 .

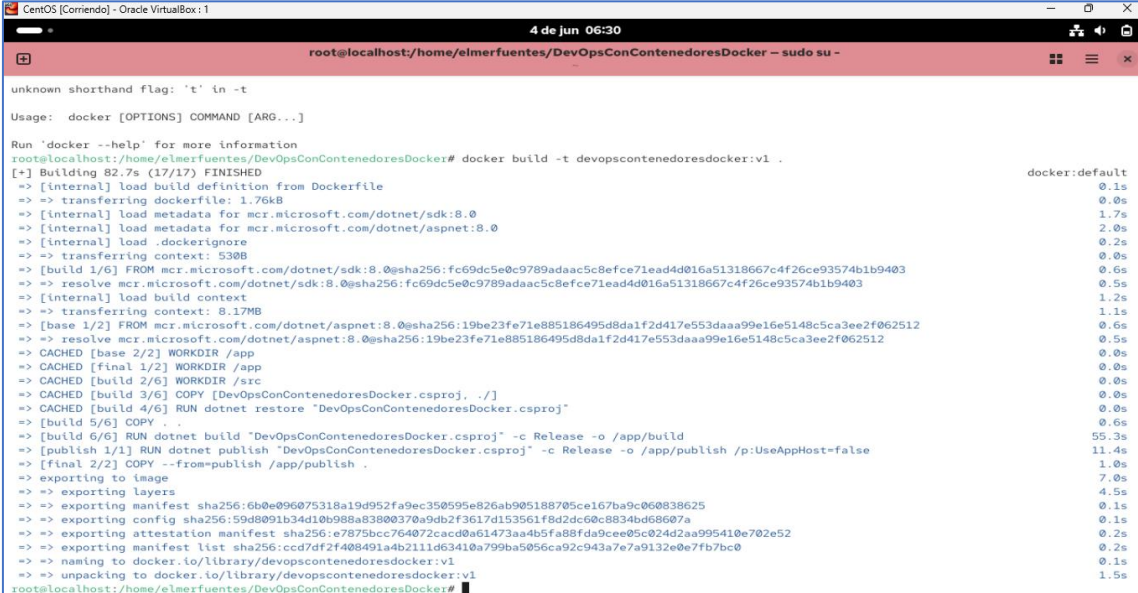
Este comando construye una imagen Docker a partir del Dockerfile del proyecto.

Descripción del comando:

docker build: construye una imagen Docker.

-t devopscontenedoresdocker:v1: asigna un nombre y una versión a la imagen.

..: indica que el contexto de construcción es la carpeta actual.



```
CentOS [Corriendo] - Oracle VM VirtualBox 1
4 de jun 06:30
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker - sudo su -

unknown shorthand flag: 't' in -t

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# docker build -t devopscontenedoresdocker:v1 .
[*] Building 82.7s (17/17) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.76kB
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:8.0
=> [internal] load .dockerignore
=> => transferring context: 530B
=> [build 1/6] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:fc69dc5e0c9789adaac5c8efce71ead4d016a51318667c4f26ce93574b1b9403
=> => resolve mcr.microsoft.com/dotnet/sdk:8.0@sha256:fc69dc5e0c9789adaac5c8efce71ead4d016a51318667c4f26ce93574b1b9403
=> [internal] load build context
=> => transferring context: 8.17MB
=> [base 1/2] FROM mcr.microsoft.com/dotnet/aspnet:8.0@sha256:19be23fe71e885186495d8da1f2d417e553daaa99e16e5148c5ca3ee2f062512
=> => resolve mcr.microsoft.com/dotnet/aspnet:8.0@sha256:19be23fe71e885186495d8da1f2d417e553daaa99e16e5148c5ca3ee2f062512
=> CACHED [base 2/2] WORKDIR /app
=> CACHED [final 1/2] WORKDIR /app
=> CACHED [build 2/6] WORKDIR /src
=> CACHED [build 3/6] COPY [DevOpsConContenedoresDocker.csproj, ./]
=> CACHED [build 4/6] RUN dotnet restore "DevOpsConContenedoresDocker.csproj"
=> [build 5/6] COPY . .
=> [build 6/6] RUN dotnet build "DevOpsConContenedoresDocker.csproj" -c Release -o /app/build
=> [publish 1/1] RUN dotnet publish "DevOpsConContenedoresDocker.csproj" -c Release -o /app/publish /p:UseAppHost=false
=> [final 2/2] COPY --from=publish /app/publish .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:6b0e096075318a19d952fa9ec350595e826ab905188705ce167ba9c060838625
=> => exporting config sha256:59d8091b34d10b988a83800370a9db2f3617d153561f8d2dc60c8834bd68607a
=> => exporting attestation manifest sha256:e7875bcc764072cadc0a61473aa4b5fa88fda9cee05c024d2aa995410e702e52
=> => exporting manifest list sha256:ccd7df2f408491a4b211d63410a799ba5056ca92c943a7e7a9132e0e7fb7bc0
=> => naming to docker.io/library/devopscontenedoresdocker:v1
=> => unpacking to docker.io/library/devopscontenedoresdocker:v1
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

2. Creación de red virtual Docker

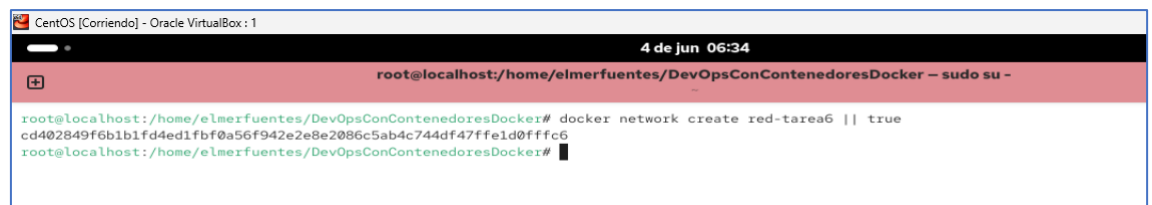
Luego se creó una red virtual para conectar el contenedor de forma controlada:

```
docker network create red-tarea6 || true
```

Descripción del comando:

`docker network create red-tarea6`: crea una red Docker llamada red-tarea6.
`|| true`: evita que el proceso falle si la red ya existe.

Esta red permite que el contenedor se ejecute dentro de una red aislada administrada por Docker.



```
CentOS [Corriendo] - Oracle VirtualBox: 1
4 de jun 06:34
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker - sudo su -
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# docker network create red-tarea6 || true
cd402849f6b1b1fd4ed1fbf0a56f942e2e8e2086c5ab4c744df47ffe1d0fffc6
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

3. Eliminación de contenedor previo

Antes de levantar un nuevo contenedor, se eliminó cualquier contenedor anterior con el mismo nombre:

```
docker rm -f devops-web 2>/dev/null || true
```

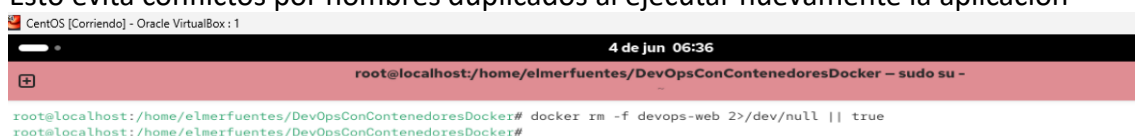
Descripción del comando:

`docker rm -f devops-web`: elimina de forma forzada el contenedor llamado devops-web, si existe.

`2>/dev/null`: oculta mensajes de error si el contenedor no existe.

`|| true`: permite continuar aunque no haya un contenedor previo.

Esto evita conflictos por nombres duplicados al ejecutar nuevamente la aplicación



```
CentOS [Corriendo] - Oracle VirtualBox: 1
4 de jun 06:36
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker - sudo su -
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# docker rm -f devops-web 2>/dev/null || true
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

4. Ejecución del contenedor

Finalmente, se levantó el contenedor con el siguiente comando:

```
docker run -d \  
  --name devops-web \  
  --network red-tarea6 \  
  -p 8080:8080 \  
  --restart unless-stopped \  
  devopscontenedoresdocker:v1
```

Descripción del comando:

docker run: ejecuta un nuevo contenedor.

-d: ejecuta el contenedor en segundo plano.

--name devops-web: asigna el nombre devops-web al contenedor.

--network red-tarea6: conecta el contenedor a la red virtual creada.

-p 8080:8080: expone el puerto 8080 del contenedor hacia el puerto 8080 de la máquina virtual.

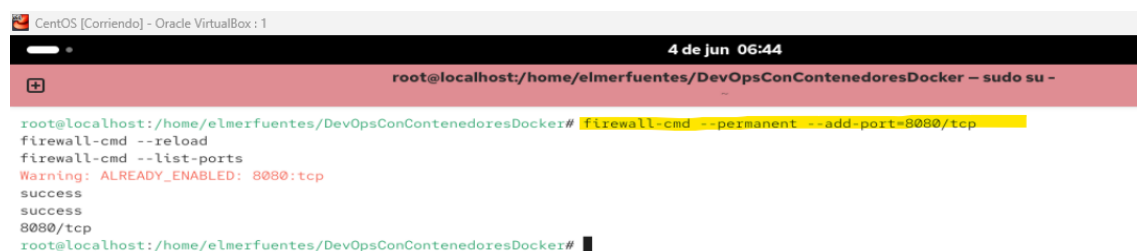
--restart unless-stopped: permite que el contenedor se reinicie automáticamente si se detiene, excepto cuando se detiene manualmente.

devopscontenedoresdocker:v1: indica la imagen Docker que se utilizará para crear el contenedor.



```
CentOS [Corriendo] - Oracle VirtualBox : 1  
4 de jun 06:42  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker - sudo su -  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# docker rm -f devops-web 2>/dev/null || true  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# docker run -d \  
  --name devops-web \  
  --network red-tarea6 \  
  -p 8080:8080 \  
  --restart unless-stopped \  
  devopscontenedoresdocker:v1  
b26a7d01e48e3e2cb1dbe5af8d482f8c3ad397cc84139bcdfe876c168808e94d  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

Permitir el tráfico a través del puerto **8080/tcp** utilizando firewallD



```
CentOS [Corriendo] - Oracle VirtualBox : 1  
4 de jun 06:44  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker - sudo su -  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# firewall-cmd --permanent --add-port=8080/tcp  
firewall-cmd --reload  
firewall-cmd --list-ports  
Warning: ALREADY_ENABLED: 8080:tcp  
success  
success  
8080/tcp  
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

5. Validación del contenedor

Para confirmar que el contenedor se encontraba en ejecución, se utilizó el comando:

```
docker ps
```

Este comando muestra los contenedores activos. Se verificó que el contenedor devops-web estuviera en estado activo y que el puerto estuviera publicado correctamente:

```
0.0.0.0:8080->8080/tcp
```

También se revisaron los logs del contenedor con:

```
docker logs devops-web
```

Esto permitió validar que la aplicación ASP.NET Core inició correctamente dentro del contenedor.

6. Prueba local desde CentOS

Para validar que la aplicación respondía dentro de la máquina virtual, se ejecutó:

```
curl -I http://localhost:8080
```

El resultado esperado es una respuesta HTTP exitosa, por ejemplo:

```
HTTP/1.1 200 OK
```

Esto confirma que la aplicación está funcionando correctamente dentro del contenedor Docker.

7. Acceso desde el navegador

Se obtuvo la dirección IP de la máquina virtual con el comando:

```
hostname -I
```

La IP identificada para acceder desde el navegador fue:

```
10.196.213.166
```

Por lo tanto, la aplicación se accedió desde el navegador utilizando la siguiente URL:

```
http://10.196.213.166:8080
```



```
CentOS [Corriendo] - Oracle VirtualBox : 1
4 de jun 06:45
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --reload
firewall-cmd --list-ports
Warning: ALREADY_ENABLED: 8080:tcp
success
success
8080/tcp
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker# hostname -I
10.196.213.166 172.17.0.1 172.18.0.1 2800:98:1b04:e82f:a00:27ff:fe1a:97c6
root@localhost:/home/elmerfuentes/DevOpsConContenedoresDocker#
```

Conexión desde Windows

